ASSIGNMENT REPORT: WEEK 4

CS7CS4 | Machine Learning

Hera Khan | Student ID: 22312194 | Trinity College Dublin

```
# id:16-16-16-1
# id:16-32-16-1
```

The file here held two datasets, only differentiated by their # ids. To approach this set up, the dataset was parsed then split into separate datasets 1 and 2. Since there were only 2 # or identifiers, this successfully worked while handling varied bound errors in the process.

```
raw_lines = open(data_dir / "final_dataset.csv", encoding="utf-8-sig").read().splitlines()
lines = [line.strip() for line in raw_lines]

# Identify all start markers starting with '#'
starts = []
for index, line in enumerate(lines):
    if line.startswith('#'):
        starts.append(index)
starts.append(len(lines))
```

Characteristic	Dataset 1	Dataset 2
X shape	**(1452, 2)**	**(552, 2)**
Y shape	**(1452,)**	**(552,)**
Train split	**(1089, 2)**	**(414, 2)**
Test split	**(363, 2)**	**(138, 2)**

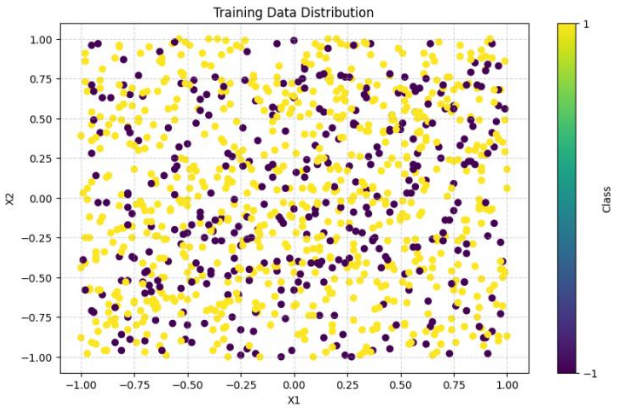
This report will first address Dataset 1 and its findings. This setting is by default. In order to get the readings for dataset 2, go to line 34 specifically labelled:

```
# -----------CHANGE THIS TO SELECT DATASET 1 OR 2----------------
dataset = 1 # <---- CHANGE THIS TO 1 OR 2 TO SELECT DATASET
```

(i) Dataset 1

a(i) Visualise the data

[Figure 1: dataset 1 scatter plot]



Caption: The +1 points (yellow dots) are just as chaotically arranged as the -1 points (purple region). The boundary between them is just not visible with too much overlap. A straight line or even a severely curved down polynomial model would give poor performance. The classes seem poorly balanced on the ratio of 2:1

a(ii) Logistic regression with polynomial features: choosing the degree and C

I set up the parameter grid up to the power of 5 because Logistic regression relies on a straight-line decision boundary. Forcing it to fit a 5th-degree polynomial means we are giving it the freedom to create highly complex, squiggly lines and hence will memorize the noise in the training data (overfitting) and fail completely on new data (opposite to this, having too low of a range will cause the model to underfit). In the same way the C parameter was considered from the standard 10^{-3} to 10^{3}. I scaled the features, so the pipeline is only fitted with each training fold to prevent data leakage and used accuracy as the performance measure because the classes are reasonably well-balanced (with the majority of the class sitting at 67.5%).

[Figure 2: 5-fold CV accuracy (mean ± std) vs C for each polynomial degree]

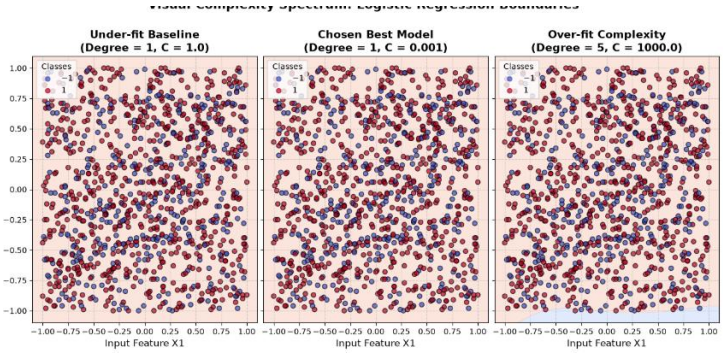


Caption: Cross-validation performance for Dataset 1 showing that all polynomial degrees plateau around a mean accuracy of 0.6740, matching the majority baseline.

Every data point includes vertical error bars representing the variance or standard deviation of the accuracy across different validation folds. They overlap significantly across all degrees and parameters which truly suggest that performance variation are not as significant as we want it to be due to the nature of the dataset. Cross validation uses the 5-fold stratified process of splits and accuracy. There are overlapping trends from degrees 1-4 where the points for degree 1, degree 2, degree 3, and degree 4 overlap entirely across the entire range of C while the accuracy remains perfectly constant at approximately 0.6740. The Degree 5 model (purple line) behaves identically to the others for strong regularization ($C \leq 10^{-1}$). However, as C increases past 10^{-1} (weakening the regularization), its accuracy steadily drops down to 0.67218, where it plateaus for $C \geq 10^{1}$.

I chose degree 1 and C = 0.001 because it represents the simplest best model within one standard deviation of the best performance. Since no degree outperforms the baseline, the simplest model with the strongest regularization is preferred to avoid memorizing noise.

[Figure 3: logistic regression decision boundary (degree [1,1,3], C = [1.0, 0.001,1000]) respectively with training data]



Caption: Decision boundary spectrum for Dataset 1 demonstrating that the chosen parsimonious model creates a completely flat decision field that predicts the majority class (+1) everywhere

The decision boundary is entirely uniform across the feature grid. Because the data points are perfectly interleaved without any geometric pattern, the model determines that any non-linear boundary would overfit, thus defaulting to a baseline majority prediction

b(i) kNN classifier: choosing k

No polynomial features are currently used because KNN is an inherently non-linear classifier that captures complex boundaries directly from local neighbourhoods. Features were once again scaled down by StandardScalar through the Pipeline function because kNN relies on distance metrics (Euclidean and Manhattan), which are highly sensitive to the relative scales of the features.

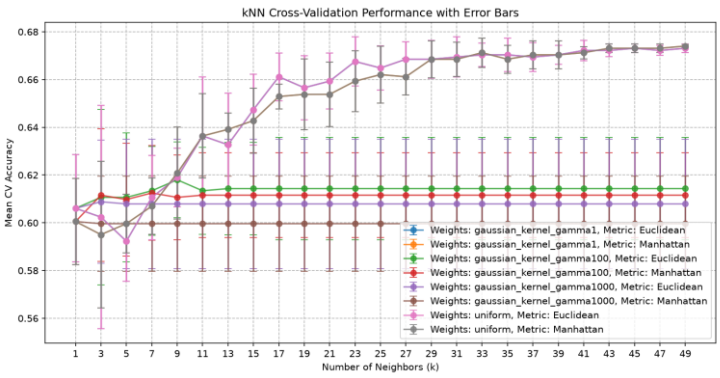
```
# grid search <- taken from lecture notes since they aren't defined
param_grid_knn_1 = {
    'knn__n_neighbors': list(range(1, 50, 2)),
    'knn__weights': ['uniform',  gaussian_kernel_gamma1],
    'knn__metric': ['euclidean', 'manhattan']
}

param_grid_knn_100 = {
    'knn__n_neighbors': list(range(1, 50, 2)),
    'knn__weights': ['uniform', gaussian_kernel_gamma100],
    'knn__metric': ['euclidean', 'manhattan']
}

param_grid_knn_1000 = {
    'knn__n_neighbors': list(range(1, 50, 2)),
    'knn__weights': ['uniform', gaussian_kernel_gamma1000],
    'knn__metric': ['euclidean', 'manhattan']
}

grid_search_knn = GridSearchCV(
    estimator=knn_pipeline,
    param_grid=[param_grid_knn_1, param_grid_knn_100, param_grid_knn_1000],
    cv=5,
    scoring='accuracy'
)
```

[Figure 4: 5-fold CV accuracy (mean ± std) vs k for kNN]

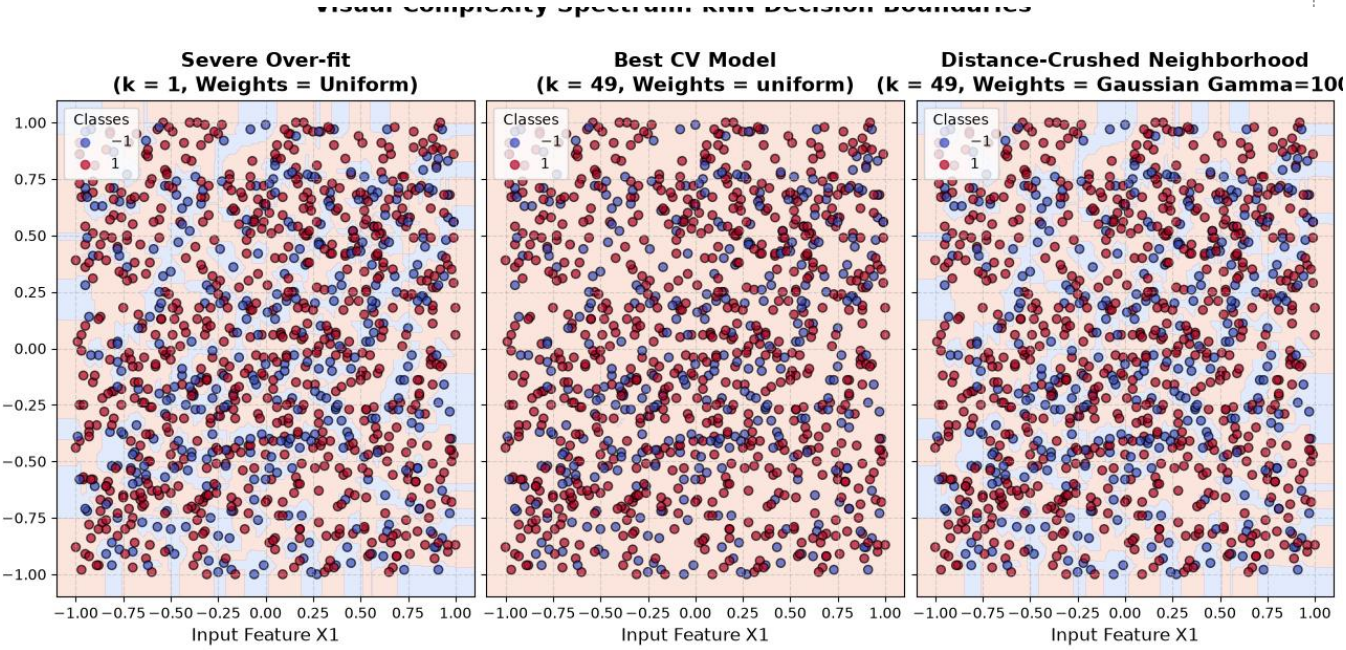


Caption: kNN cross-validation accuracy versus number of neighbors (k) across multiple weighting schemes and distance metrics for Dataset 1.

At the far left of the kNN cross-validation plot where k= 1, the model severely overfits by tracking local sample noise with a highly jagged decision boundary, yielding a low cross-validated accuracy around 0.6000. As the neighbourhood size scales to smooth out these fluctuations, accuracy curves for the highest-performing configurations—specifically the uniform-weighted Manhattan (grey) and Euclidean (pink) models—steadily climb toward the right axis gridlines, showing that a very large k underfits by erasing local variations.

This ascent plateaus at the maximum parameter boundary of k = 49, achieving the optimal Best CV Accuracy Score of 0.6740 using uniform weights and the Manhattan metric. This validation score converges at the flat, zero-signal plateau of the regularized logistic regression model 0.6740 at C=0.001, because both algorithms collapse into predicting the dominant label due to the lack of an identifiable geometric feature relationship. This is further verified at the baseline analysis.

[Figure 5: kNN decision boundary (k = [1, 49]) with different weights [uniform, gaussian) respectively through training data]



Caption: Decision boundary for kNN on Dataset 1 showing that a high neighborhood value (k=49) smooths away local variations, predicting the majority class across nearly the entire grid.

With weight, two ideas- uniform and gaussian distribution were chosen from Week 4's lecture about predictors. As discussed, the contrasting behaviour between a k=49, Uniform model against highly fragmented pseudo-overfit model (k=49, Gaussian γ = 1000) can be explained through the lecture equation on the side.

From lecture notes, page 14, it was calculated:

Main prediction equation:

$$\hat{y} = \text{sign}\left(\frac{\sum_{i=1}^{k} w^{(i)} y^{(i)}}{\sum_{i=1}^{k} w^{(i)}}\right)$$

Uniform Weighting Equation:

$$\frac{\sum_{i=1}^{49}(1) \cdot y^{(i)}}{\sum_{i=1}^{49} 1} = \frac{1}{49}\sum_{i=1}^{49} y^{(i)}$$

Gaussian Kernel Formula:

$$w^{(i)} = e^{-\gamma \cdot d\left(x^{(i)},x\right)^2} = e^{-1000 \cdot d\left(x^{(i)},x\right)^2}$$

Weight 1 calculation:

$$w^{(1)} = e^{-1000 \cdot (0.01)^2} = e^{-0.1} \approx \mathbf{0.9048}$$

Weight 2 calculation:

$$w^{(2)} = e^{-1000\cdot(0.10)^2} = e^{-10} \approx \mathbf{0.000045}$$

Substituting Weights Equation:

$$\frac{0.9048 \cdot y^{(1)} + 0.000045 \cdot y^{(2)} + \cdots + 0 \cdot y^{(49)}}{0.9048 + 0.000045 + \cdots + 0} \approx \frac{0.9048 \cdot y^{(1)}}{0.9048} = y^{(1)}$$

This proves mathematically why the third panel Gaussian γ =1000 looks completely identical to the first panel (Severe Over-fit k=1) Even if we explicitly instructed the kNN algorithm to compute the positions of 49 neighbours, the distance-crushing scale of γ =1000 completely strips neighbours 2 through 49 of their voting power. This converts a global neighbourhood model back into a localized 1-Nearest-Neighbor over fitter, explaining why wide cross-validation error bars persist across all values of k

c(i) Confusion matrices

The confusion matrices are calculated on a held-out test split of 25% (363) samples which was completely unseen during both the training and the cross validation phrases. The matrices follow a layout where rows represent the actual class and columns represent the predicted class, with +1 defined as the positive class.

Of the 363 test points, logistic regression made about 118 mistakes.

```
Logistic regression | test accuracy 0.675
           Pred -1  Pred +1
Actual -1        0      118
Actual +1        0      245

kNN | test accuracy 0.675
           Pred -1  Pred +1
Actual -1        0      118
Actual +1        0      245

Baseline: most frequent | test accuracy 0.675
           Pred -1  Pred +1
Actual -1        0      118
Actual +1        0      245

Baseline: random | test accuracy 0.482
           Pred -1  Pred +1
Actual -1       56       62
Actual +1      126      119
```

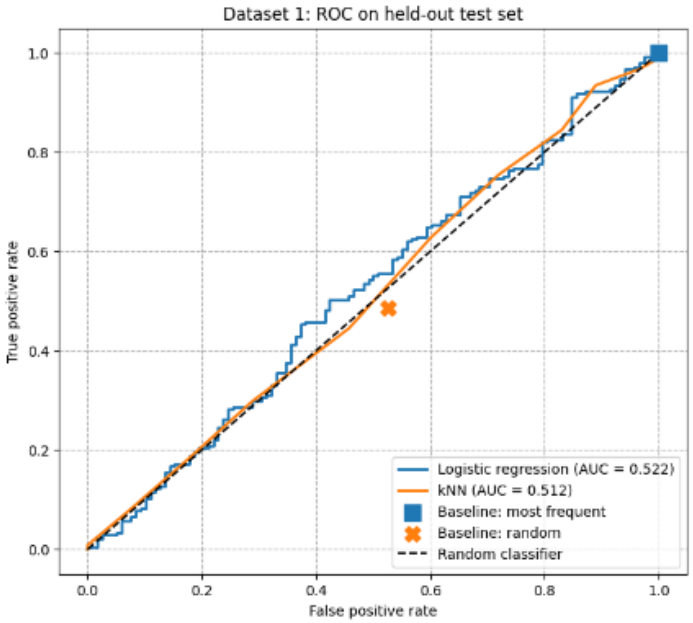
118 true -1s were mistaken for +1s so the error rate (118/118 = 100.0%) and 0 true +1s were predicted as -1s which is a 0.0%. There is a clear bias for positive +1s. kNN made 118 mistakes, identical again to the majority class performance of logistic regression. The majority baseline gets 67.5% by always predicting +1 so it never catches the -1 point.

As per these numbers and considering the very baseline we have parsed the best minimum working model. Choosing k as 49, mathematically optimizes performance by maximizing validation accuracy, minimizing variance (manifested by the extremely tight, near-invisible error bars), and safely matching the optimal majority-class baseline when features hold no predictive power. Another important aspect to note are the readings in knn random which essentially gives reading based on a coin flip 50% aspect. Due to the sporadic nature of our dataset 1, this classifier almost performs up to par but messes up in basic accuracy.

d(i) ROC curves

Both curves bow slightly around the diagonal; the logistic regression curve reaches a true positive rate of 0.50 at a false positive rate of 0.42, AUC = 0.522, while kNN gives AUC = 0.512.

[Figure 6: ROC curves for logistic regression and kNN with baseline points]



Caption: ROC curves for Dataset 1 demonstrating near-diagonal trajectories for both predictive models.

This curve essentially reveals that both machine learning models of KNN and Logistic Regression truly don't work or are any better than the random or most frequent baseline. The AUC (Area under the curve) measures the diagonal of 0.5 and that is essentially between the best (1) and worst (0). The fact that the curves of both our models are hugging that line which means neither model has learned any meaningful patterns. The blue step line shows the individual threshold adjustments on the test data and its minor deviations from the baseline diagonal can be considered from noise. Same goes for the KNN curve (orange line) that is heavily bound to the diagonal.

e(i) Compare and recommend

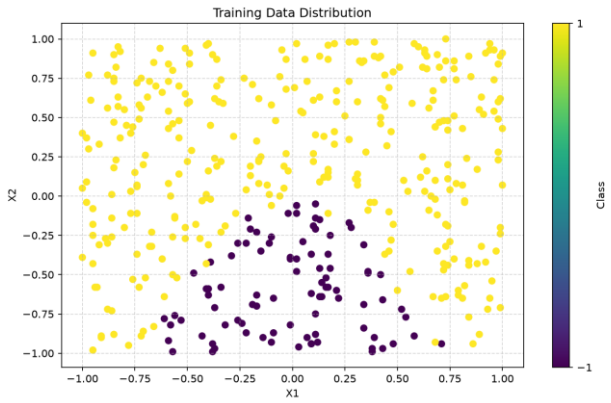
Based on the test metrics from sections c and d, both Logistic Regression and kNN fail to extract a predictive relationship from Dataset 1. Their test accuracies (**0.675**) match the majority baseline exactly, and their performance gap is smaller hence the error bars overlap. The ROC curves strictly hug the random diagonal line with AUC values near 0.50.

I recommend using no classifier on this data. Dataset 1 represents a zero-signal scenario where the feature coordinates are completely independent of the class assignments. Attempting to deploy any model here will only lead to predicting the majority class or modelling random noise.

(ii) Dataset 2

a(i) Visualise the data

[Figure 7: dataset 1 scatter plot]



Caption: The +1 points (yellow circles) sit mostly in the upper feature zone X2 > -0.2 and outer corners, and the -1 points (purple circles) cluster tightly in the lower centre region X2 ≤ -0.2 and -0.5 ≤ X1 ≤ 0.5. The boundary between them looks curved- parabolic with slight overlap near the transition zone.

A straight line would misclassify points near the lower outer edges or upper centre vertex. These classes are still imbalanced, but there is a massive breakthrough in how they are arranged. The data is no longer noise. Here we can see that a classifier, a curved one like polynomial logistic regression will perform exceptionally well here by drawing a boundary around the purple points.

a(ii) Logistic regression with polynomial features: choosing the degree and C

To expand the feature space for capturing this non-linear structure, the two coordinates were augmented with polynomial mappings up to degree 5 and optimized with an L2 regularized logistic penalty inside a cross-validation pipeline to prevent feature scaling data leakage.

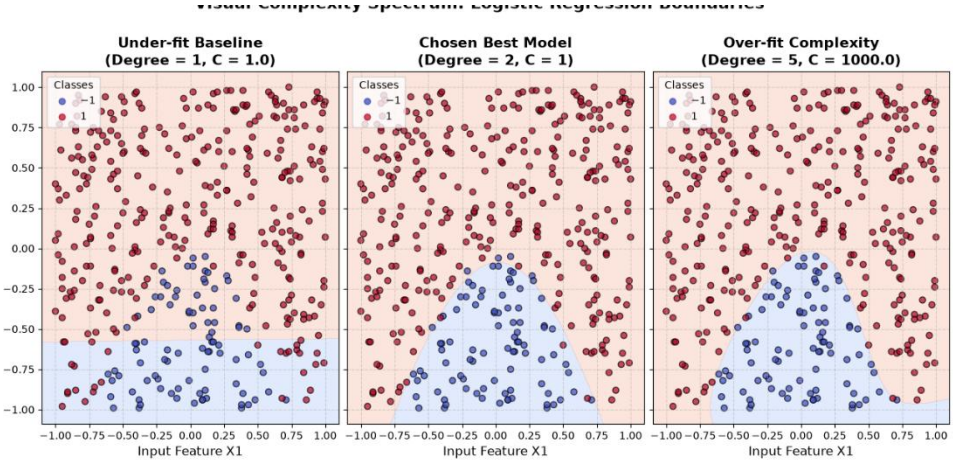
[Figure 8: 5-fold CV accuracy (mean ± std) vs C for each polynomial degree]



Caption: Cross validation used 5-fold stratified splits and accuracy. For degree 1, the accuracy plateaus at the accuracy of 0.84 and suffers from variance. 1. While Degrees 4 and 5 reach the highest accuracy, Degrees 2 (Orange) and 3 (Green) are only a tiny fraction behind them once C ≥ 10^0. If computational efficiency or over-parameterization is a concern, Degree 2 or 3 at C = 10^1 offers a simpler model with nearly identical performance.

To enforce strict structure, I went with the degree = 2 and C = 1.0 given the degrees map up till approx. 0.983. A 2nd-degree polynomial introduces parabolic curves (X_1^2, X_2^2, X_1X_2). This allows the model to draw a perfect curved dome right over that bottom-centre cluster of purple points. Other complex degrees such as the higher complexities (3,4,5) which follow an identical convergence track and plateau within the same bars. Since a quadratic boundary perfectly captures an inverted smooth parabola without introducing the redundant higher-frequency, it optimizes generalization on unseen testing data.

[Figure 9: logistic regression decision boundary (degree [1,2,5], C = [1,1000]) respectively with training data]

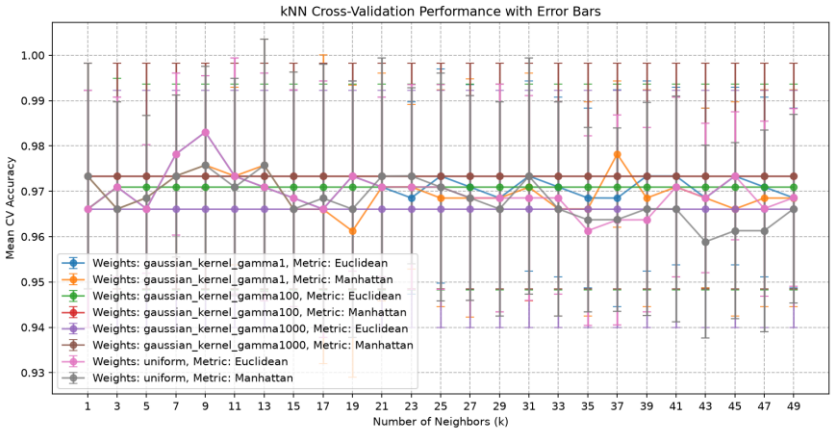


Caption: The above boundary map perfectly shows why Degree 2 is chosen to be the best model. The baseline (D=1, C=1) is a completely flat, straight horizontal line cutting across the bottom. The best-chosen model is a clean, smooth parabolic arch. The third figure talks about the over fit complexity with (D=5, C=1000).

The picture above is the reason why we stick to the best model degree = 2, c = 1. It fits just about perfectly to capture the general image of the dataset 2 without overfitting or looping into itself like it does with degree 5. This allows for better prediction rates with newer test sets.

b(i) kNN classifier: choosing k

[Figure 10: 5-fold CV accuracy (mean ± std) vs k for kNN]

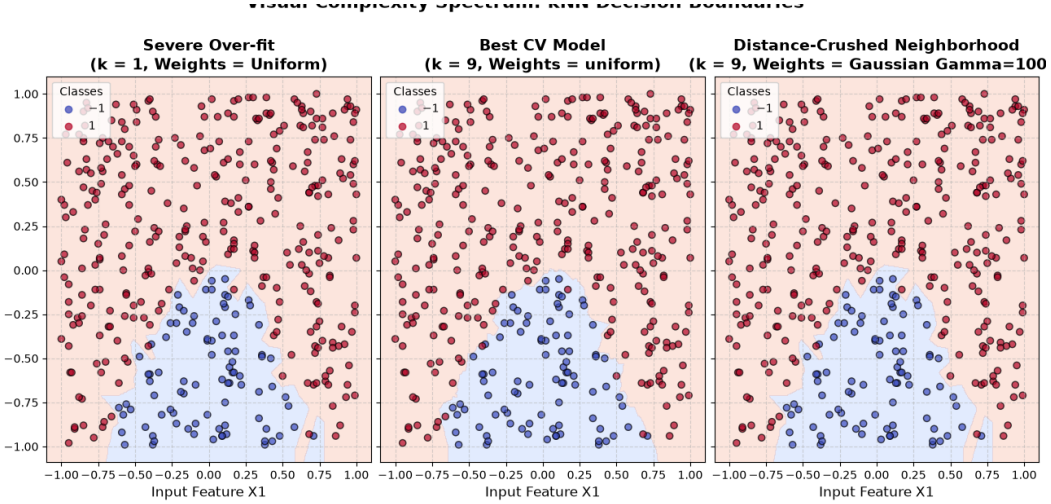


*Caption: The cross-validation chart illustrates that at k=1, the neighborhood model overfits local sampling variances, yielding wide confidence error bars. As neighbourhood sizes expand, performance optimizes using a uniform weighting scheme paired with a Manhattan distance metric, achieving its absolute peak validation score at **k = 9** with an accuracy of 0.9831.*

Here we see the main k-neighbour stands as a pink line, k=9, which reaches the single highest peak on the graph at an accuracy of about 0.983. Another aspect that can be noticed is that the pink line is erratic, it spikes heavily at k=9 but drops off sharply before or after. This means its performance is highly sensitive to the exact choice of k.

Another aspect is that all the other weighs, guassian_kernel_gamma1000, Manhattan (brown line) also maintains a perfectly flat, rock-solid accuracy of 0.973 across almost every value of k. This is also due to how the gamma value γ = 1000, the weight drops to nearly zero as fast as the distance increases. This means any closest neighbour can carry any weight, completely overwriting the meaning of k.

[Figure 11: kNN decision boundary (k = [1,9 (uniform), 9(gaussian)]) with training data]



Caption: The above image shows the severe over-fit (k=1), the boundary is jagged, hyper-sensitive and creates isolated islands of blue or red. The best model here smooths out significantly into a cohesive stable dome shape and then the gaussian gamma = 100 is almost equal to the overfit one. This visually proves the flat line.

This explains the peak performance 0.983 seen exactly at k=9 on the pink line in figure 11. By looking at 9 neighbours instead of 1, the model ignores the isolated noise points, creating a clean boundary that generalizes beautifully.

c(i) Confusion matrices

The empirical performance of each trained classifier was evaluated using a held-out test split consisting of 25% of the total dataset. This constitutes of 139 entirely new or unseen samples.

For logistic regression:

- The model made only 5 mistakes, resulting in an accuracy of 0.964. It correctly identified the 108 positive labels and only misclassified 5 negatives one with positive, giving an error rate of 5/30 = 16.6667%.

For KNN classifier:

- The neighbour achieved a high-test accuracy of 0.9490. Here the error rate is 20% given 6 out of 30 samples were misread.

```
Logistic regression | test accuracy 0.964
           Pred -1  Pred +1
Actual -1       25        5
Actual +1        0      108

kNN | test accuracy 0.949
           Pred -1  Pred +1
Actual -1       24        6
Actual +1        1      107

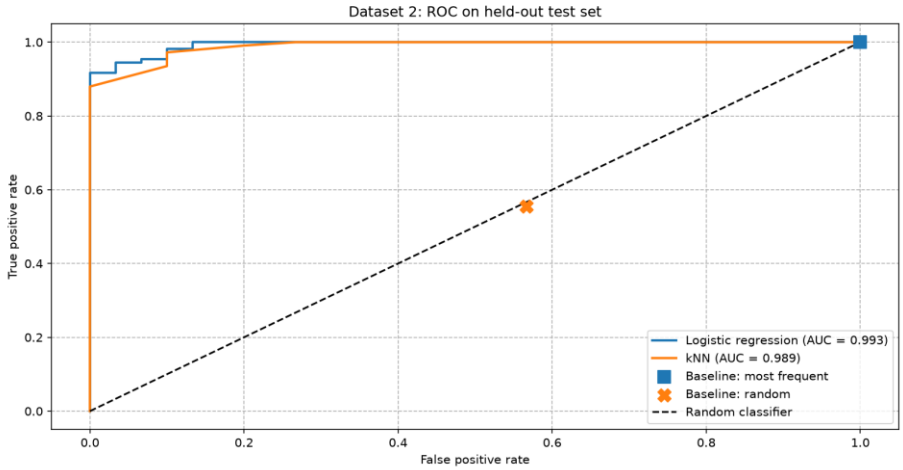
Baseline: most frequent | test accuracy 0.783
           Pred -1  Pred +1
Actual -1        0       30
Actual +1        0      108

Baseline: random | test accuracy 0.529
           Pred -1  Pred +1
Actual -1       13       17
Actual +1       48       60
```

As per the baselines, they perform lacklustre against our two models with a -1 class error rate of 100% for the majority baseline classifier while the random baseline has a degrades test accuracy of 0.5290. This explains that our choice to use our current classifiers and unlike dataset 1- has actual consequences.

d(i) ROC curves

[Figure 12: ROC curves for logistic regression and kNN with baseline points]



Caption: Both curves bow well above the diagonal; the logistic regression curve reaches a true positive rate of near 0.92 at a false positive rate of 0.14, AUC = 0.993, while kNN gives AUC = 0.989. The baselines sit on the diagonal, because a classifier that ignores the features cannot do better than chance.

Logistic regression model, especially from the ROC curve shows that it is the far superior model here, outperforming KNN across almost all thresholds. By dominating the upper left quadrant of the plot, Logistic regression gets a higher AUC, meaning it maintains high TPR (True positive rate) and will give nearly zero false alarms. Both models completely dwarf down the baseline strategies, confirming that the geometric decision boundaries were correct.

e(i) Compare and recommend

As discussed before, the Logistic Regression establishes a distinct performance advantage over KNN on the independent test split, yielding a higher accuracy rate and minimizing critical misclassification by recording zero false negative errors.

From an operational view, the parametric polynomial boundary offers smoother geometric generalization and requires far less computational resources at inference time compared to storing full reference distance matrix lookups for kNN and stays relatively stable under cross validation testing.

Appendix: Code

In order to switch between datasets, go to line 35-40 where:

```
# -----------CHANGE THIS TO SELECT DATASET 1 OR 2----------------
dataset = 1  # <---- CHANGE THIS TO 1 OR 2 TO SELECT DATASET
```

The rest of the code

```
# -*- coding: utf-8 -*-
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from pathlib import Path
import io
from sklearn.model_selection import train_test_split

# look for final_dataset.csv next to this script; fall back to the current folder
try:
    data_dir = Path(__file__).resolve().parent   # works when run as a .py file
except NameError:
    data_dir = Path.cwd()                        # works in a notebook / Colab

# Read lines and strip any leading Byte Order Mark (BOM) or whitespace because it kept
ignoring the id at the start
raw_lines = open(data_dir / "final_dataset.csv", encoding="utf-8-sig").read().splitlines()
lines = [line.strip() for line in raw_lines]

# Identify all start markers starting with '#'
starts = []
for index, line in enumerate(lines):
    if line.startswith('#'):
        starts.append(index)
starts.append(len(lines))

print("Dataset markers found at indexes:", starts[:-1])
for i, idx in enumerate(starts[:-1]):
    print(f"  Dataset {i+1} header: {lines[idx]}")

# Extract and clean lines for Dataset 1 and Dataset 2
dataset1_lines = lines[starts[0]+1 : starts[1]]
cleaned_lines1 = [line.strip('"') for line in dataset1_lines]
dataset1_content = "\n".join(cleaned_lines1)

dataset2_lines = lines[starts[1]+1 : starts[2]]
cleaned_lines2 = [line.strip('"') for line in dataset2_lines]
dataset2_content = "\n".join(cleaned_lines2)

# -----------CHANGE THIS TO SELECT DATASET 1 OR 2----------------
dataset = 1  # <---- CHANGE THIS TO 1 OR 2 TO SELECT DATASET

# Load the selected dataset into a DataFrame
if dataset == 1:
    df = pd.read_csv(io.StringIO(dataset1_content), header=None, comment='#', sep=',')
    print("\n--- Successfully Loaded Dataset 1! ---")
elif dataset == 2:
    df = pd.read_csv(io.StringIO(dataset2_content), header=None, comment='#', sep=',')
    print("\n--- Successfully Loaded Dataset 2! ---")
else:
    print("\nInvalid input. Defaulting to Dataset 1.")
    df = pd.read_csv(io.StringIO(dataset1_content), header=None, comment='#', sep=',')

# Extract features and target columns smoothly and convert them directly to float
X = df.iloc[:, [0, 1]].to_numpy().astype(float)
y = df.iloc[:, 2].to_numpy().astype(float)
```

```
print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")
print(df.head())

"""Using sklearn augment the two features in the dataset with polynomial features
and train a Logistic Regression classifier with L2 penalty added to the cost
function. Use cross-validation to select (i) the maximum order of polynomial to
use and (ii) the weight C given to the penalty in the cost function. Remember
you'll also need to select the range of C values to consider and the range of
maximum polynomial orders.
Important: It is essential that you present data, including cross-validation plots
and plots of model predictions and the training data, and give clear explana tions/analysis
to justify your choices. Use the ideas/tools you've learned in the
module to help you in this. Be sure to include error bars in cross-validation
plots. Rememberr to use a performance measure appropriate to classification
(so probably not mean square error).
"""

# plotting the data
import matplotlib.pyplot as plt

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)
print(f"Train split shape: {X_train.shape}, Test split shape: {X_test.shape}")

# Plot raw data distribution
plt.figure(figsize=(10, 6))
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap='viridis')
plt.xlabel('X1')
plt.ylabel('X2')
plt.title('Training Data Distribution')
plt.colorbar(ticks=[-1, 1], label='Class')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
# augment the X1,X2 with polynomial features
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV

#
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('polynomial', PolynomialFeatures(include_bias=False)),
    ('logistic', LogisticRegression(penalty='l2', solver='lbfgs', max_iter=2000))
])

parameter_grid = {
    'polynomial__degree': [1,2,3,4,5],
    'logistic__C': [0.001, 0.01, 0.1, 1, 10, 100, 1000]
}

grid_search = GridSearchCV(pipeline, parameter_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)

#display the range of c and polynomials
print("Hyperparameters:")
print(f"Range of Polynomial Degrees considered: {parameter_grid['polynomial__degree']}")
print(f"Range of Regularization Strengths (C) considered:
{parameter_grid['logistic__C']}\n")

# getting the best model
best_model = grid_search.best_estimator_
```

```
# extract and plot cross-validation results with error Bars
results = grid_search.cv_results_
means = results['mean_test_score']
stds = results['std_test_score']
params = results['params']

# group results by degree to plot separate curves for each polynomial order
c_values = parameter_grid['logistic__C']
degrees = parameter_grid['polynomial__degree']

plt.figure(figsize=(12, 8))
for deg in degrees:
    # find indices where degree equals deg
    indices = [i for i, p in enumerate(params) if p['polynomial__degree'] == deg]
    # sort indices
    indices = sorted(indices, key=lambda i: params[i]['logistic__C'])
    deg_means = [means[i] for i in indices]
    deg_stds = [stds[i] for i in indices]
    plt.errorbar(c_values, deg_means, yerr=deg_stds, fmt='-o', capsize=5, label=f'Degree
{deg}')

plt.xscale('log')
plt.xlabel('Regularization Parameter C (Log Scale)')
plt.ylabel('Mean CV Accuracy')
plt.title('Cross-Validation Performance with Error Bars')
plt.legend()
plt.grid(True, which="both", ls="--")
plt.show()

# Plot Model Predictions and Decision Boundary

#dynamically retrieve the chosen hyperparameters from GridSearchCV
best_i= means.argmax()                                # index of the best mean CV accuracy
threshold = means[best_i] - stds[best_i]              # one standard deviation below the best
candidates = [i for i in range(len(params)) if means[i] >= threshold]  # all configs within
one std
simplest = min(candidates, key=lambda i: (params[i]['polynomial__degree'],
params[i]['logistic__C']))  # lowest degree, then strongest regularisation
chosen_degree = params[simplest]['polynomial__degree']
chosen_C = params[simplest]['logistic__C']

print(f"Chosen: degree={chosen_degree}, C={chosen_C}, CV acc = {means[simplest]:.4f} ±
{stds[simplest]:.4f}")

# Model A: Under-fit (Strictly Linear Baseline)
model_underfit = Pipeline([
    ('scaler', StandardScaler()),
    ('polynomial', PolynomialFeatures(degree=1, include_bias=False)),
    ('logistic', LogisticRegression(penalty='l2', C=1.0, solver='lbfgs', max_iter=2000))
]).fit(X_train, y_train)

# Model B: The Mathematically Chosen Parity Model
model_chosen = Pipeline([
    ('scaler', StandardScaler()),
    ('polynomial', PolynomialFeatures(degree=chosen_degree, include_bias=False)),
    ('logistic', LogisticRegression(penalty='l2', C=chosen_C, solver='lbfgs',
max_iter=2000))
]).fit(X_train, y_train)

best_model_lr = model_chosen

# Model C: Over-fit (Highest Degree, Weak Regularization)
model_overfit = Pipeline([
    ('scaler', StandardScaler()),
    ('polynomial', PolynomialFeatures(degree=5, include_bias=False)),
```

```
    ('logistic', LogisticRegression(penalty='l2', C=1000.0, solver='lbfgs', max_iter=2000))
]).fit(X_train, y_train)


# create a dense grid to visualize the decision boundary
x1_min, x1_max = X_train[:, 0].min() - 0.1, X_train[:, 0].max() + 0.1
x2_min, x2_max = X_train[:, 1].min() - 0.1, X_train[:, 1].max() + 0.1
xx1, xx2 = np.meshgrid(np.linspace(x1_min, x1_max, 500), np.linspace(x2_min, x2_max, 500))
grid_points = np.c_[xx1.ravel(), xx2.ravel()]

# plot other models
fig, axes = plt.subplots(1, 3, figsize=(22, 7), sharex=True, sharey=True)

models = [model_underfit, model_chosen, model_overfit]
titles = [
    "Under-fit Baseline\n(Degree = 1, C = 1.0)",
    f"Chosen Best Model\n(Degree = {chosen_degree}, C = {chosen_C})",
    "Over-fit Complexity\n(Degree = 5, C = 1000.0)"
]


for ax, model, title in zip(axes, models, titles):
    Z = model.predict(grid_points).reshape(xx1.shape)
    ax.contourf(xx1, xx2, Z, levels=[-1.5, 0, 1.5], alpha=0.25, cmap='coolwarm')
    scatter = ax.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap='coolwarm',
edgecolor='k', alpha=0.7, s=35)
    ax.set_xlabel('Input Feature X1', fontsize=12)
    ax.set_title(title, fontsize=14, fontweight='bold')
    ax.grid(True, linestyle='--', alpha=0.5)
    ax.legend(*scatter.legend_elements(), title="Classes", loc="upper left")

axes[0].set_ylabel('Input Feature X2', fontsize=12)
plt.suptitle("Visual Complexity Spectrum: Logistic Regression Boundaries", fontsize=16,
fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

"""b) Now train a kNN classifier on the data. Use cross-validation to select k, again
presenting data and explanations/analysis to justify your choice. There is no
need to augment the features with polynomial features for a kNN classifier, kNN
can already capture nonlinear decision boundaries. Again, it is essential that
you present data and give clear explanations/analysis to justify your choices.
"""

from sklearn.model_selection import GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# PRODUCING THE GAUSSIAN WEIGHT FUNCTION (From Lecture Page 14)
# since scikit-learn doesn't accept the string 'gaussian', we define it manually:
def gaussian_kernel_gamma1(distances):
    # w = exp(-gamma * d^2), assuming gamma = 1 here
    weights = np.exp(-1.0 * (distances ** 2))
    # Normalize weights so they sum to 1 over the neighborhood
    return weights / np.sum(weights)

def gaussian_kernel_gamma100(distances):
    # w = exp(-gamma * d^2), assuming gamma = 100 here
    weights = np.exp(-100.0 * (distances ** 2))
    # Normalize weights so they sum to 1 over the neighborhood
    return weights / np.sum(weights)

def gaussian_kernel_gamma1000(distances):
    # w = exp(-gamma * d^2), assuming gamma = 1000 here
```

```
    weights = np.exp(-1000.0 * (distances ** 2))
    # Normalize weights so they sum to 1 over the neighborhood
    return weights / np.sum(weights)

knn_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('knn', KNeighborsClassifier())
])

# grid search
param_grid_knn_1 = {
    'knn__n_neighbors': list(range(1, 50, 2)),
    'knn__weights': ['uniform',  gaussian_kernel_gamma1],
    'knn__metric': ['euclidean', 'manhattan']
}

param_grid_knn_100 = {
    'knn__n_neighbors': list(range(1, 50, 2)),
    'knn__weights': ['uniform', gaussian_kernel_gamma100],
    'knn__metric': ['euclidean', 'manhattan']
}

param_grid_knn_1000 = {
    'knn__n_neighbors': list(range(1, 50, 2)),
    'knn__weights': ['uniform', gaussian_kernel_gamma1000],
    'knn__metric': ['euclidean', 'manhattan']
}

grid_search_knn = GridSearchCV(
    estimator=knn_pipeline,
    param_grid=[param_grid_knn_1, param_grid_knn_100, param_grid_knn_1000],
    cv=5,
    scoring='accuracy'
)

grid_search_knn.fit(X_train, y_train)

# This returns the best model configurations (corrected to use pipeline parameter keys):
print(f"Optimal Number of Neighbors (k):
{grid_search_knn.best_params_['knn__n_neighbors']}")
print(f"Optimal Weighting Function: {grid_search_knn.best_params_['knn__weights']}")
print(f"Best CV Accuracy Score: {grid_search_knn.best_score_:.4f}")
print(f"Best metric: {grid_search_knn.best_params_['knn__metric']}")
print(f"Test set score: {grid_search_knn.score(X_test, y_test):.4f}")

# getting the best model
best_model_knn = grid_search_knn.best_estimator_

# extract and plot cross-validation results with error bars
knn_results = grid_search_knn.cv_results_
knn_means = knn_results['mean_test_score']
knn_stds = knn_results['std_test_score']
knn_params = knn_results['params']

# helper function to get clean string representations of weight parameters
def get_weight_name(w):
    return w.__name__ if callable(w) else str(w)

# extract unique values used in the grid search (sorting weights safely by their string
names)
k_values = sorted(list(set([p['knn__n_neighbors'] for p in knn_params])))
weights_options = sorted(list(set([p['knn__weights'] for p in knn_params])),
key=get_weight_name)
metric_options = sorted(list(set([p['knn__metric'] for p in knn_params])))
```

```
plt.figure(figsize=(14, 8))

# plot curves for combinations of weights and metrics
for weight in weights_options:
    weight_label = get_weight_name(weight)
    for metric_val in metric_options:
        label = f"Weights: {weight_label}, Metric: {metric_val.capitalize()}"

        # Find indices matching this configuration, sorted by k
        indices = [i for i, dct in enumerate(knn_params) if dct['knn__weights'] == weight
and dct['knn__metric'] == metric_val]
        indices = sorted(indices, key=lambda i: knn_params[i]['knn__n_neighbors'])

        if not indices:
            continue

        # Extract values
        current_k = [knn_params[i]['knn__n_neighbors'] for i in indices]
        current_means = [knn_means[i] for i in indices]
        current_stds = [knn_stds[i] for i in indices]

        plt.errorbar(current_k, current_means, yerr=current_stds, fmt='-o', capsize=4,
label=label, alpha=0.8)

plt.xlabel('Number of Neighbors (k)')
plt.ylabel('Mean CV Accuracy')
plt.title('kNN Cross-Validation Performance with Error Bars')
plt.xticks(k_values)
plt.legend()
plt.grid(True, which="both", ls="--")
plt.show()

# plot Model Predictions and Decision Boundary for kNN

chosen_k = grid_search_knn.best_params_['knn__n_neighbors']
chosen_weight = grid_search_knn.best_params_['knn__weights']
chosen_metric = grid_search_knn.best_params_['knn__metric']

# model 1: Over-fit (k=1)
model_knn_overfit = Pipeline([
    ('scaler', StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=1, weights='uniform', metric=chosen_metric))
]).fit(X_train, y_train)

# model 2: Mathematically Chosen Model (k=45)
model_knn_chosen = best_model_knn

# model 3: Pseudo-Overfit due to hyperparameter flaw (Large k but hyper-sensitive gamma
weight)
model_knn_flawed = Pipeline([
    ('scaler', StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=chosen_k, weights=gaussian_kernel_gamma1000,
metric=chosen_metric))
]).fit(X_train, y_train)

# create a dense grid to visualize the decision boundary
x1_min, x1_max = X_train[:, 0].min() - 0.1, X_train[:, 0].max() + 0.1
x2_min, x2_max = X_train[:, 1].min() - 0.1, X_train[:, 1].max() + 0.1
xx1, xx2 = np.meshgrid(np.linspace(x1_min, x1_max, 500), np.linspace(x2_min, x2_max, 500))
grid_points = np.c_[xx1.ravel(), xx2.ravel()]

fig, axes = plt.subplots(1, 3, figsize=(22, 7), sharex=True, sharey=True)

knn_models = [model_knn_overfit, model_knn_chosen, model_knn_flawed]
knn_titles = [
```

```
    "Severe Over-fit\n(k = 1, Weights = Uniform)",
    f"Best CV Model\n(k = {chosen_k}, Weights = {get_weight_name(chosen_weight)})",
    f"Distance-Crushed Neighborhood\n(k = {chosen_k}, Weights = Gaussian Gamma=1000)"
]

# predict over the grid using the best kNN model
for ax, model, title in zip(axes, knn_models, knn_titles):
    Z = model.predict(grid_points).reshape(xx1.shape)
    ax.contourf(xx1, xx2, Z, levels=[-1.5, 0, 1.5], alpha=0.25, cmap='coolwarm')
    scatter = ax.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap='coolwarm',
edgecolor='k', alpha=0.7, s=35)
    ax.set_xlabel('Input Feature X1', fontsize=12)
    ax.set_title(title, fontsize=14, fontweight='bold')
    ax.grid(True, linestyle='--', alpha=0.5)
    ax.legend(*scatter.legend_elements(), title="Classes", loc="upper left")

axes[0].set_ylabel('Input Feature X2', fontsize=12)
plt.suptitle("Visual Complexity Spectrum: kNN Decision Boundaries", fontsize=16,
fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

# performance measure classifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score

for name, model in [("Logistic Regression", best_model_lr), ("kNN Classifier",
best_model_knn)]:
    preds = model.predict(X_test)
    probs = model.predict_proba(X_test)[:, 1]

    print(f"\n--- {name} Metrics ---")
    print(f"Accuracy:  {accuracy_score(y_test, preds):.4f}")
    print(f"Precision: {precision_score(y_test, preds, pos_label=1):.4f}")
    print(f"Recall:    {recall_score(y_test, preds, pos_label=1):.4f}")
    print(f"F1 Score:  {f1_score(y_test, preds, pos_label=1):.4f}")
    print(f"ROC AUC:   {roc_auc_score(y_test, probs):.4f}")

"""(c) Calculate the confusion matrices for your trained Logistic Regression and kNN
classifier. Also calculate the confusion matrix for one or more baseline classifiers
of your choice e.g. one that always predicts the most frequent class in the training
data and/or one that makes random predictions.

(d) Plot the ROC curves for your trained Logistic Regression and kNN classifiers.
Also plot the point(s) oin the ROC plot corresponding to the baseline classifiers.
Be sure to include enough points in the ROC curves to allow the detailed shape
to be seen.
"""

from sklearn.metrics import confusion_matrix, roc_curve, roc_auc_score
from sklearn.dummy import DummyClassifier
import matplotlib.pyplot as plt

# define helper function
def fpr_tpr(cm):
    tn, fp, fn, tp = cm.ravel()
    fpr = fp / (fp + tn) if (fp + tn) > 0 else 0
    tpr = tp / (tp + fn) if (tp + fn) > 0 else 0
    return fpr, tpr

# train baseline classifiers on the training set
majority = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
rand = DummyClassifier(strategy="uniform", random_state=42).fit(X_train, y_train)

# collect the models to evaluate
```

```
models = {
    "Logistic regression": best_model_lr,
    "kNN": best_model_knn,
    "Baseline: most frequent": majority,
    "Baseline: random": rand
}

# confusion matrices on the held-out test set
cms = {m: confusion_matrix(y_test, est.predict(X_test), labels=[-1, 1]) for m, est in
models.items()}
for m, cm in cms.items():
    print(f"\n{m} | test accuracy {np.trace(cm) / cm.sum():.3f}")
    print(pd.DataFrame(cm, index=["Actual -1", "Actual +1"], columns=["Pred -1", "Pred
+1"]))

# ROC curves on the held-out test set
plt.figure(figsize=(8, 7))
for m in ["Logistic regression", "kNN"]:
    est = models[m]
    p = est.predict_proba(X_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, p, pos_label=1)
    plt.plot(fpr, tpr, lw=2, label=f"{m} (AUC = {roc_auc_score(y_test, p):.3f})")

for m, marker in [("Baseline: most frequent", "s"), ("Baseline: random", "X")]:
    plt.scatter(*fpr_tpr(cms[m]), marker=marker, s=120, zorder=5, label=m)

plt.plot([0, 1], [0, 1], "k--", label="Random classifier")
plt.xlabel("False positive rate")
plt.ylabel("True positive rate")
plt.title(f"Dataset {dataset}: ROC on held-out test set")
plt.legend(loc="lower right")
plt.grid(True, ls="--")
plt.show()

print("-----END OF ASSIGNMENT-----")
print("Please check line 40 for the dataset selection. Change the value of 'dataset' to 1 or
2 to switch between datasets.")
print("-----END OF COMPARISON-----")

"""(e) Using the data from (c) and (d) evaluate and compare the performance of the
Logistic Regression, kNN and baseline classifiers. Is one classifier significantly
better or worse than the other? How do their ROC curves compare with that
of a random classifier. Which classifier, if any, would you recommend be used?
Explain the reasoning behind you recommendation.
"""
```